PROJECT GUIDE & PRESENTATION COMPANION

AI-Based User and Entity Behaviour Analytics System

How the system works end to end, why each design choice was made, what the results mean, and how to present and defend it in front of a panel.



Student	Amaan Saify
Roll number	2023BCSE07AED025
Coordinator	Dr. Rathnakar Achary
Stack	Python • pandas • scikit-learn • SQLite • Streamlit • Plotly
Date	October 2026

CONTENTS

What is in this guide

Read sections 1-3 first: they give you the whole story in a few pages. Sections 4-9 explain each stage in depth. Sections 10-14 are for the presentation itself.

1	The project in one page	elevator pitch, headline results
2	UEBA basics and key terms	what UEBA is, why it is needed
3	Architecture and end-to-end workflow	the 9 steps from logs to alert
4	Stage 1 - Log capture	4 log streams, simulated company, attacks
5	Stage 2 - Behavioural features	turning events into daily numbers
6	Stage 3 - Behaviour profiles	baselines, z-scores, adaptive learning
7	Stage 4 - Detection (3 layers)	statistics, ML ensemble, threat rules
8	Stage 5 - Risk scoring and alerts	fusion formula, threshold, decay
9	Case study: one attack, start to finish	INC-01 brute force, with real numbers
10	Dashboard tour	the 6 pages and what to show
11	Evaluation and results	metrics, robustness, ablation
12	Code map, tech stack, how to run	which file does what
13	Presenting: talk outline and demo script	10-minute talk, 5-minute demo
14	Viva questions and answers	25 likely questions
15	Limitations, future work, cheat sheet	numbers to remember

SECTION 1

The project in one page

A Python system that watches how every user and computer in an organisation normally behaves, notices when that behaviour changes in a dangerous way, and raises a prioritised, explained security alert.

The problem statement (as assigned)

> *"Develop a Python-based User and Entity Behavior Analytics (UEBA) system that captures authentication, system-access, network, and resource-usage logs, builds behavioral profiles for users and devices, identifies deviations from normal behavior, and generates risk-based security alerts."*

What was built, mapped to each requirement

Requirement	What the system does
Captures 4 log types	Authentication (logins, VPN, SSH, RDP), system access (files, privileged commands), network (domains, ports, bytes), resource usage (CPU, memory, disk). Simulated for a 48-person company; also captures **live** data from your own laptop and can read the public **CERT insider-threat dataset**.
Behavioural profiles for users and devices	44 daily features per user and 19 per device. Each entity gets a personal baseline (its own normal) and a peer baseline (its department's normal). Profiles keep learning over time.
Identifies deviations	Three independent layers: robust statistics (z-scores), an unsupervised ML ensemble (Isolation Forest, Local Outlier Factor, Autoencoder), and 20 threat rules mapped to MITRE ATT&CK.
Risk-based alerts	All signals are fused into a 0-100 risk score, adjusted for how critical the account is. Alerts carry a severity, plain-English reasons and MITRE technique IDs, and are shown in a Streamlit dashboard.

Headline results

11/11
planted attacks detected, all on day 1

0.96
precision, recall and F1 (main run)

1
false alert in 30 days, 108 entities

0.95
mean F1 over 5 different simulated companies

SAY IT LIKE THIS

"Traditional security tools look for known signatures, so they miss attackers who log in with stolen but valid passwords, and insiders who already have access. My system learns what is normal for each user and each device, and alerts when behaviour changes. It combines statistics, three unsupervised machine-learning models and MITRE-mapped rules into one 0-100 risk score, and every alert explains itself. Tested on 60 days of logs from a simulated 48-person company with 11 planted attacks, it caught all 11 on the day they happened, with one false alert in a month."

SECTION 2

UEBA basics and key terms

Understand this page and you can answer most "why" questions from the panel.

Why signature-based security is not enough

Antivirus, firewalls and intrusion-detection systems mostly match **known bad patterns**: a malware hash, a blacklisted IP, an exploit signature. They struggle with three very common situations:

- **Stolen credentials.** The attacker logs in with a real username and password. Every log line looks legitimate.
- **Insider threats.** An employee already has access and slowly copies data out before leaving.
- **Novel (zero-day) attacks.** There is no signature yet.

In all three, the clue is **behaviour**: logging in at 3 a.m. from Moscow, reading 90 files instead of the usual 15, uploading 3 GB to a site never used before. **UEBA (User and Entity Behaviour Analytics)** is the class of security tool that learns normal behaviour and flags the change. Gartner named the category; commercial examples include Microsoft Sentinel UEBA, Splunk UBA and Exabeam.

Users and entities

User = a person's account (e.g. `pooja.sharma`). **Entity** = anything else that behaves: laptops, servers, IP addresses, applications. This project profiles **48 users and 60 devices (48 laptops + 12 servers) = 108 entities**. Profiling devices matters because some attacks show up on a machine, not an account (e.g. cryptomining malware pins a laptop's CPU).

Key terms

Term	Meaning in this project
Baseline / profile	What is normal for an entity, learned from its own history (e.g. "usually logs in 09:00-18:00, uploads ~1 MB/day").
Entity-day	The unit the system scores: one entity on one day. 108 entities × 30 detection days ≈ 2,363 entity-days that had activity.
Feature	A number describing one entity-day, e.g. failed logins = 45, MB uploaded = 401.
Anomaly / deviation	A feature value far from the baseline.
z-score	How many "typical spreads" today's value is from normal. z = 0 is normal; z ⩾ 3 counts as unusual here.
Peer group	Users in the same department. Lets the system judge brand-new users and spot behaviour odd for the role.
Unsupervised learning	ML trained only on normal data, with no attack labels. Needed because real organisations rarely have labelled attacks.
Risk score	0-100 number combining all evidence, used to prioritise what an analyst looks at first.
False positive	An alert on harmless behaviour. Too many causes "alert fatigue", the main reason real UEBA deployments fail.
MITRE ATT&CK	Public catalogue of attacker techniques with IDs (e.g. T1110 Brute Force). Tagging alerts with IDs is industry practice.
SOC	Security Operations Centre: the team that reads alerts. The dashboard is built for them.

SECTION 3

Architecture and end-to-end workflow

Data flows left to right through five stages. Every stage is a separate Python module, so each can be explained, tested and replaced on its own.

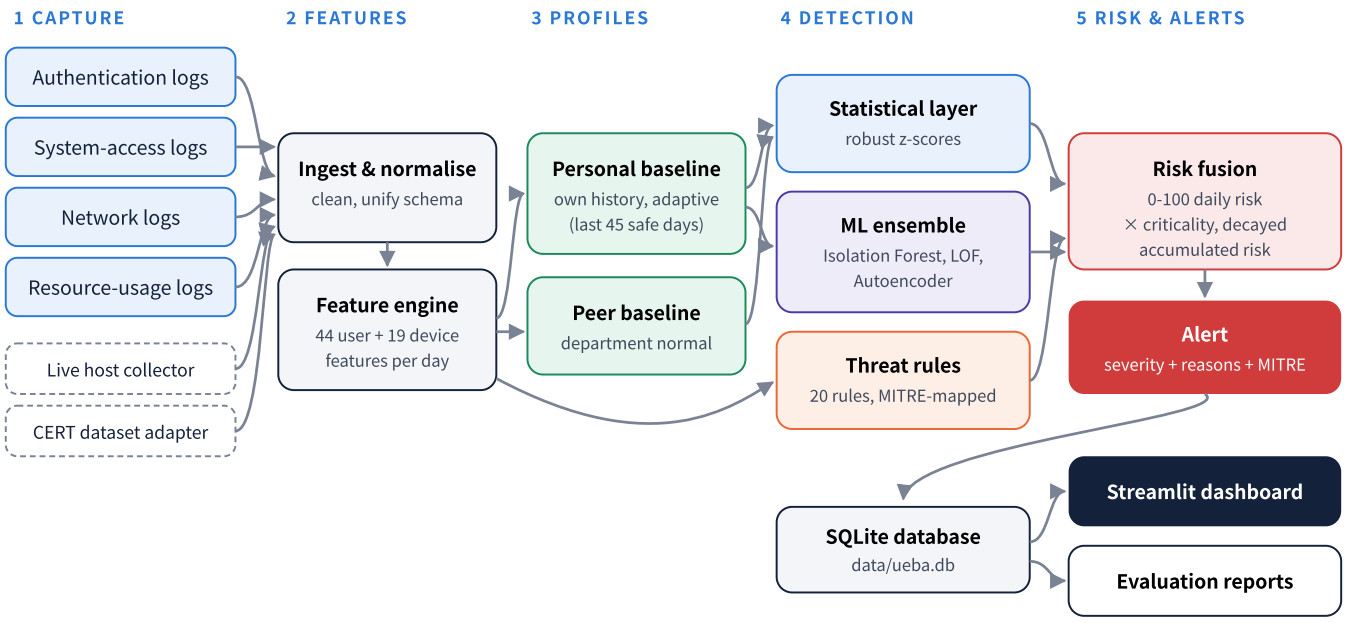


Figure 3.1 - System architecture. Dashed boxes are optional inputs (live laptop data, CERT benchmark). All results are stored in one SQLite file that the dashboard and the evaluation module read.

The 9-step workflow (what happens when you run `python run_pipeline.py`)

1. **Generate / collect logs**
   60 days of logs for 108 entities, ~205,000 events, with 11 planted attacks.
   generator.py • collector.py • cert_loader.py
2. **Ingest and normalise**
   Read CSV/JSON, fix types, lower-case names, merge into one event timeline.
   ingest.py
3. **Build daily features**
   Summarise each entity-day into numbers (logins, MB uploaded, new hosts ...).
   features.py
4. **Learn baselines**
   Days 6-30 (attack-free) teach each entity's and each department's normal.
   profiles.py
5. **Score deviations**
   Each detection day: z-scores vs. baseline, ML anomaly scores, rule checks.
   detectors.py • pipeline.py
6. **Fuse into risk**
   One 0-100 daily risk per entity-day, plus an accumulated risk that decays.
   risk.py
7. **Raise alerts**
   Risk ⩾ 45 → alert with severity, reasons and MITRE IDs.
   risk.py
8. **Store and display**
   Everything saved to SQLite; dashboard reads it.
   ingest.py • dashboard/app.py
9. **Evaluate**
   Compare alerts with the known attacks; write metrics and charts.
   evaluate.py • experiments.py

SECTION 4 • STAGE 1

Log capture

Real company logs are confidential, and real data never says which events were attacks. So the project simulates a realistic company with **known** attacks hidden inside, which makes accurate measurement possible.

The simulated company

48 employees in 6 departments (Engineering 14, Sales 10, Finance 8, HR 6, IT-Admin 5, Management 5), each with their own laptop, plus 12 servers (domain controller, file share, ERP, CRM, Git, backups ...). Every person gets **individual habits**: start time and length of workday, office or remote city, which servers and files they use, how much data they upload, how busy their CPU is. Some IT admins work afternoon shifts. Weekends are quiet, about 5% of weekdays are leave days, and people occasionally mistype passwords or upload a large file legitimately.

Timeline: 60 days (1 Jul - 29 Aug 2026). Days 1-5 warm-up, days 6-30 baseline (no attacks), days 31-60 detection window (attacks planted).

The four log streams (real rows from the data)

Authentication - `auth.csv` (9,347 events)

timestamp	user	src_device	src_ip	dest_host	auth_type	result	city	country
2026-07-20 09:21:50	pooja.sharma	LT-SAL-034	10.10.4.44	LT-SAL-034	interactive	success	Bengaluru	IN
2026-08-03 03:10:00	pooja.sharma	UNKNOWN	185.220.101.47	vpn-gw01	vpn	failure	Moscow	RU

System access - `access.csv` (75,295 events)

timestamp	user	device	resource	action	sensitivity
2026-07-01 07:41:29	ishaan.pillai	LT-FIN-019	/shares/finance/report_18.py	write	internal
2026-08-28 03:08:00	maria.singh	UNKNOWN	/shares/hr/payroll/salaries_2026.xlsx	read	restricted

Network - `network.csv` (85,432 events)

timestamp	user	device	dst_ip	port	bytes_out	bytes_in	domain
2026-07-01 07:30:12	ishaan.pillai	LT-FIN-019	142.58.229.74	443	15,140	221,136	slack.com
2026-08-03 03:32:05	pooja.sharma	UNKNOWN	142.216.252.56	443	400,000,000	20,000	transfer.sh

Resource usage - `resource.csv` (35,409 hourly samples)

timestamp	device	user	cpu_pct	mem_pct	disk_io_mb	process_count
2026-07-01 07:28:01	LT-FIN-019	ishaan.pillai	11.5	56.5	11.82	139

Live capture from a real machine

`ueba/collector.py` uses the **psutil** library to read the actual CPU, memory, process count and open network connections of the computer running the dashboard, in the same schema. It proves the "capture" stage works on genuine telemetry. Demo it on the *Live host monitor* page.

CERT insider-threat benchmark

`ueba/cert_loader.py` converts Carnegie Mellon's public CERT r4.2 dataset (logon, USB, file, web, email logs of 1,000 simulated employees with labelled insiders) into this schema. It is tested on a format-identical sample; the full 10-15 GB dataset must be downloaded separately.

The 11 planted attacks (ground truth)

Each attack is written into the logs exactly as it would appear in reality, and recorded in `ground_truth.csv` so detection can be scored. Each covers a different MITRE ATT&CK technique and a different department.

ID	Scenario	What happens in the logs	MITRE
INC-01	Brute force + takeover	45 failed VPN logins from a Tor exit node at 03:10, then success, files read, 400 MB to transfer.sh	T1110
INC-02	Impossible travel	Login in Bengaluru, then from Amsterdam 70 minutes later	T1078
INC-03	Off-hours access	HR user logs in at 02:20, reads 60 confidential files incl. payroll, uploads to WeTransfer	T1078
INC-04	Data exfiltration	Engineer reads 90 files + production API keys, uploads 3.3 GB to mega.nz	T1567
INC-05	Slow insider	5 days of growing Dropbox uploads (120 MB → 1.3 GB) plus job-portal visits	T1567.002
INC-06	Privilege escalation	Sales user RDPs into the domain controller and runs admin commands on restricted data	T1068
INC-07	Lateral movement	SSH/RDP into 12 never-before-seen hosts within 50 minutes	T1021
INC-08	Cryptomining	Laptop CPU at ~96% for 3 days, traffic to a Monero mining pool on port 3333	T1496
INC-09	Password spray	30 failed logins on an admin account from Tor, no success	T1110
INC-10	Travel + exfiltration	Login from Lagos 70 min after Bengaluru, then 1.2 GB upload	T1078/T1567
INC-11	Admin account abuse	Admin login from Frankfurt at 03:05, 4 AD accounts created, security log cleared	T1136/T1070

In total, 28 entity-days are labelled as attacks: 17 user-days and 11 device-days, because when an attack runs from a person's own laptop the laptop is labelled too.

Four harmless oddities, planted on purpose

A detector that only sees clean normal days and obvious attacks would look unrealistically perfect. So the data also contains suspicious-looking but **innocent** behaviour, which is **not** labelled as an attack:

Late-night deadline
An engineer works until 1 a.m. once.

Business trip
A salesperson logs in from Singapore for 3 days over VPN.

Big backup
An IT admin sends 20 GB to the internal backup server.

New project
An HR user opens 11 files from another department.

Why this matters in the viva
These test whether the system can tell "unusual" from "dangerous". Three of the four are correctly ignored. The "new project" case raises the system's **only false alert** (risk 49): it genuinely is unusual, and an analyst would close it in seconds after reading the reasons.

SECTION 5 • STAGE 2

Behavioural features

Machine-learning models cannot read raw log lines. The feature engine turns all of one entity's events on one day into a fixed list of numbers, its "behaviour fingerprint" for that day.

User features (44 per user per day)

Group	Features	Catches
Authentication (14)	logins, failed logins, max failures in any 10 min, failure-burst-then-success, source IPs, *new* source IPs, hosts, *new* hosts, SSH/RDP sessions, logins from unregistered devices, logins at unusual hours, countries, *new* countries, impossible-travel hops	brute force, stolen credentials, lateral movement, travel
System access (12)	accesses, confidential/restricted accesses, restricted only, privilege escalations, deletions, distinct and *new* resources, *new* sensitive resources, off-hours accesses, account creations, log-clearing actions, USB events	data theft, privilege abuse, covering tracks
Network (12)	connections, MB up / down, largest single upload, domains, *new* domains, MB to *new* external domains, MB to cloud storage, *new* ports, mining-pool connections, job-site visits, internal hosts	exfiltration, mining, insider flight risk
Resource (6)	average and peak CPU, hours above 85% CPU, memory, disk I/O, active hours	cryptomining, malware

Device features (19 per device per day) mirror these from the machine's point of view: logins *into* the device, distinct and new users on it, its network uploads and new domains, and its CPU / memory / disk / process counts.

Three feature ideas worth explaining

"New" = never seen before

A host, country, domain or file counts as new only if this entity has **never** used it on any earlier day. Computed causally (only looking backwards), exactly like a live system. First-week days are warm-up, because everything is new then.

Unusual hour is personal

For each login, the system checks how often **this user** logged in at that hour on all previous days. Below 2% probability means unusual. 3 a.m. is normal for a night-shift admin but alarming for a 9-to-6 salesperson.

Impossible travel

For consecutive successful logins, distance (haversine formula on latitude/longitude) ÷ time. Over 500 km at more than **900 km/h** (faster than an airliner) is impossible. Bengaluru → Amsterdam = 7,700 km in 70 min ≈ 6,600 km/h.

Example: the same user, a normal day vs. the attack day

Feature (pooja.sharma)	Usual (median)	3 Aug 2026 (attack)
failed logins	0	**45**
max failures in 10 minutes	0	**45**
new countries	0	**1** (Russia)
impossible-travel hops	0	**1**
logins from unregistered devices	0	**2**
file accesses at unusual hours	0	**11**
MB uploaded	1.1	**401**
MB to a never-used external domain	0	**400**

SECTION 6 • STAGE 3

Behaviour profiles

A profile answers "what is normal for this entity?" Each feature gets a centre (typical value) and a spread (how much it normally varies). Deviations are measured in units of that spread.

Robust statistics, not mean and standard deviation

Behaviour data is spiky: most days a person uploads 1 MB, but once a month they upload a 60 MB presentation. Mean and standard deviation are dragged around by such spikes, so the system uses **robust** statistics:

```
# for every entity and every feature
centre = median(history)
spread = max( 1.4826 × MAD,                  # MAD = median absolute deviation
              (P95 − median) / 1.645,        # respects the person's own occasional peaks
              floor )                        # avoids dividing by ~0

z = (today − centre) / spread                # clipped to [−10, +10]
```

- **1.4826 × MAD** makes the MAD comparable to a standard deviation for normal data, without being affected by outliers.
- **The P95 term** was added after testing: without it, every legitimate monthly upload was flagged. With it, someone who uploads 60 MB a few times a month is not flagged for doing it again. It cut false positives by roughly two-thirds.
- **The floor** handles features that are always zero (e.g. impossible travel), whose spread would otherwise be 0.

Worked example

pooja.sharma's upload volume: median ≈ 1.1 MB, P95 ≈ 47.9 MB, so spread ≈ (47.9 − 1.1)/1.645 ≈ 28.5 MB. On the attack day she uploaded 401 MB: z ≈ (401 − 1.1)/28.5 ≈ **14**, clipped to 10. Anything above z = 3 counts as a deviation.

Two baselines per entity

Personal baseline - "unusual for you?"

Learned from the entity's own history. Catches changes in one person's habits, even if the new behaviour is normal for someone else.

Peer baseline - "unusual for your role?"

Learned from the whole department. Gives new employees a sensible baseline and catches behaviour odd for the job (e.g. a Sales user running admin commands).

Adaptive learning with quarantine

People change: new projects, new tools. A baseline frozen on day 30 would raise more and more false alerts. So during detection, each day is compared with the entity's **last 45 accepted days**, and the profile keeps learning. To stop an attacker from slowly "teaching" the system that the attack is normal, a day is **quarantined** (never learned) if its risk is ⩾ 60 or it matched any threat rule.

SAY IT LIKE THIS

"The profile is a sliding window that keeps learning normal change, but suspicious days are quarantined, so an attacker cannot poison the baseline."

Analyst view: each user also gets a readable profile, e.g. pooja.sharma: 09:00-18:00 • IN / Bengaluru • hosts LT-SAL-034, sharepoint01 • ~5 MB uploaded/day • ~25 file accesses/day.

SECTION 7 • STAGE 4

Detection: three independent layers

Each layer outputs a score between 0 and 1. They are deliberately different, so each covers the others' blind spots. This is called **defence in depth**.

1 • Statistical

Takes the 3 largest z-scores above 3 and combines them into one 0-1 score (personal), and the same against peers.
Strength: simple and explainable, produces the "401 MB vs usual 1" sentences.
Weakness: judges features one at a time.

2 • Machine learning

Three unsupervised models look at all 44 z-scores **together**, trained only on normal days.
Strength: catches unusual combinations; no attack labels needed.
Weakness: harder to explain on its own.

3 • Threat rules

20 if-then signatures of known attacker techniques, each mapped to MITRE ATT&CK.
Strength: very precise for known attacks.
Weakness: blind to anything not written as a rule.

The ML ensemble in plain words

Model	Idea	Analogy
Isolation Forest	Builds 300 random trees that keep splitting the data. Unusual points are isolated after very few splits; normal points need many. Short path = anomaly.	Finding the one person in a crowd wearing a red hat takes one question.
Local Outlier Factor	Compares how crowded a point's neighbourhood is (25 nearest neighbours) with how crowded its neighbours' neighbourhoods are. A point in a much emptier region is an outlier.	A house in the middle of an empty field, far from any town.
Autoencoder	A small neural network (44 → 16 → 6 → 16 → 44) trained to compress and rebuild normal days. It rebuilds normal days well and abnormal days badly; high reconstruction error = anomaly.	Someone who has only seen English letters trying to copy Chinese characters.

Calibration: the three models output numbers on different scales. Each is rescaled against its own scores on normal training days (1.0 = the 99th percentile of normal), mapped to 0-1, and the three are averaged. **Why an ensemble?** Each model has blind spots; averaging three different approaches is more stable than trusting one.

The 20 threat rules

ID	Rule	Fires when	Weight	MITRE
R01	Brute-force success	⩾10 failures in 10 min, then a success	0.80	T1110
R02	Brute-force / spray attempt	⩾10 failures in 10 min, no success	0.45	T1110
R03	Impossible travel	implied speed > 900 km/h	0.70	T1078
R04	New country	login from a country never seen for the user	0.25	T1078
R05	Privilege escalation by non-admin	sudo / runas by a non-IT account	0.80	T1068
R06	Account manipulation	New-ADUser, useradd, net user /add ...	0.50	T1136
R07	Log tampering	wevtutil cl, Clear-EventLog ...	0.80	T1070
R08	Upload to new external domain	⩾100 MB to a never-used domain	0.55	T1048
R09	Upload to personal cloud	⩾50 MB to mega, Dropbox, WeTransfer ...	0.45	T1567.002
R10	Crypto-mining traffic	mining-pool domain or Stratum ports	0.80	T1496
R11	Lateral movement burst	⩾5 never-seen hosts in one day	0.55	T1021
R12	Off-hours sensitive access	⩾10 accesses at the user's rare hours	0.35	T1078

R13	Unregistered device	login from a device not in the inventory	0.25	T1078
R14	Job-site browsing	job portals, non-HR staff (insider indicator)	0.10	-
R15	First-time restricted access	⩾3 new sensitive files by a user who never had restricted access	0.30	T1213
D01-D05	Device rules: mining host, CPU > 85% for ⩾ 6 h, upload to new domain, upload to cloud, ⩾ 10 failed logins into the host		0.35-0.80	T1496, T1048, T1567, T1110

Several rules on one day combine as *1 − Π(1 − weight)*: any strong rule pushes the score up, more rules push it further, and it never exceeds 1.

SECTION 8 • STAGE 5

Risk scoring and alerts

Analysts cannot read 2,363 entity-days. Risk scoring turns all evidence into one number from 0 to 100, so the most dangerous cases come first.

The fusion formula

```
A = 1 − (1 − 0.28·stat) × (1 − 0.10·peer) × (1 − 0.30·ml) × (1 − rules)

daily_risk = 100 × min(1, A × criticality)

criticality:  IT-Admin, servers 1.3 · Finance, Management 1.2 · HR 1.1 · others 1.0
```

This is a **probabilistic OR**: treat each weighted layer as a chance of "this is bad"; the chance that *at least one* is right is 1 minus the chance all are wrong. Any strong signal raises risk, agreeing signals raise it further, and it never exceeds 100. **Criticality** reflects impact: a compromised admin or finance account does more damage than a sales account, so the same behaviour scores higher.

Why the alert threshold is 45

> The two baseline layers on their own can reach at most 0.953 each. Even then, A = 1 − (1 − 0.28×0.953)(1 − 0.10×0.953) = 0.337, and × 1.3 for an admin = **43.8**, just under 45. So **statistics alone can never raise an alert**: the ML ensemble or a threat rule must agree. This "corroboration" design is what removed most false positives, such as a Finance user's first ever 50 MB upload.

Severity

Risk	Severity	Meaning for the analyst
⩾ 80	CRITICAL	Investigate now
60-79	HIGH	Investigate today
45-59	MEDIUM	Review in queue

Accumulated risk: catching slow attacks

```
entity_risk(today) = min(100, entity_risk(yesterday) × 0.5^(days / 3)  +  max(0, daily_risk − 15))
```

Daily risk above a noise floor of 15 is added to a running total that halves every 3 days. A slow insider who stays a little suspicious every day builds up risk; a single odd day fades away. If accumulated risk reaches 60 without a recent alert, a **sustained-risk** alert is raised as a safety net.

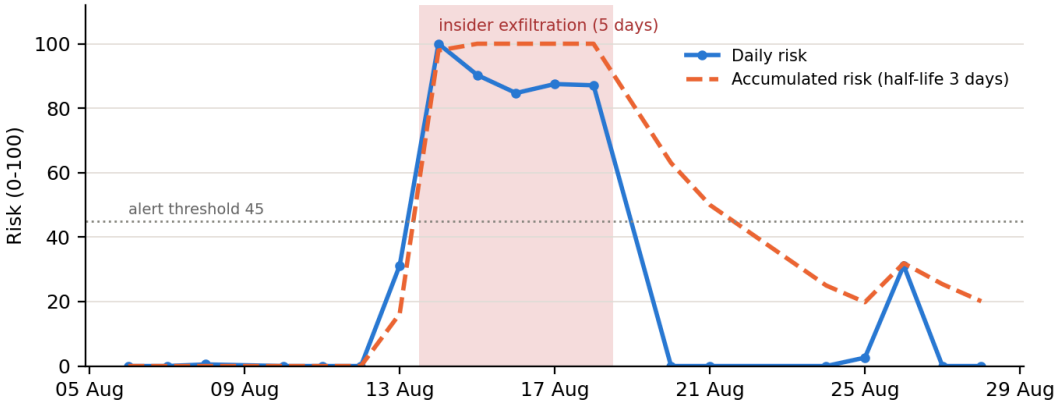


Figure 8.1 - INC-05 slow insider (sana.pillai). Daily risk jumps on each of the 5 exfiltration days; accumulated risk stays at 100, then halves every 3 days once the behaviour stops.

What every alert contains

ID • date • entity and department • title (the strongest rule, or "Behavioural anomaly") • severity • risk score • **reasons in plain English** (rule messages + the biggest deviations, e.g. "401 MB uploaded vs usual 1 (388x usual) - also unusual vs peers") • MITRE technique IDs • which layers triggered it. **Explainability** matters: an analyst will not act on a black-box number.

SECTION 9

Case study: one attack, start to finish

INC-01, 3 August 2026. The account of pooja.sharma, a Bengaluru salesperson who normally works 09:00-18:00, is brute-forced from a Tor exit node. Use this walk-through when the panel asks "show me how it works".

1 Raw logs

time	event	detail
03:10:00-03:16:36	auth vpn failure ×45	from 185.220.101.47 (Tor), Moscow, device UNKNOWN
03:17:05	auth vpn success	same IP: the password was guessed
03:20:05	auth kerberos success	into sharepoint01
03:22:05 ...	access read ×25	/shares/sales/... files
03:32:05	network upload	400 MB to transfer.sh
09:35:04	auth interactive success	the real Pooja logs in from Bengaluru on her own laptop

2 Features for the day

failed logins 45 • max failures in 10 min 45 • fail-then-success 1 • new countries 1 • impossible travel 1 (last login in Moscow at 03:20, next in Bengaluru at 09:35: 5,862 km in 6.25 h ≈ 938 km/h, faster than the 900 km/h limit) • unregistered-device logins 2 • off-hours file accesses 11 • MB uploaded 401 • MB to new domain 400.

3 Profile comparison (z-scores)

failed logins, failures in 10 min, off-hours access, MB uploaded and largest upload all hit the cap of **z = 10**. Her baseline for all of these is ~0.

4 Layer scores

Layer	Score	Why
Personal baseline	0.95	five features at z = 10
Peer baseline	0.95	no salesperson behaves like this
ML ensemble	0.75	LOF 1.00, Autoencoder 1.00, Isolation Forest 0.25
Threat rules	0.995	7 rules: R01 brute-force success (0.80), R03 impossible travel (0.70), R08 new-domain upload (0.55), R09 cloud upload (0.45), R12 off-hours (0.35), R04 new country (0.25), R13 unregistered device (0.25)

5 Risk fusion

```
A = 1 − (1 − 0.28×0.953)(1 − 0.10×0.953)(1 − 0.30×0.75)(1 − 0.9946)
  = 1 − 0.733 × 0.905 × 0.775 × 0.0054  =  0.997
daily_risk = 100 × 0.997 × 1.0 (Sales)  =  99.7   →  CRITICAL
```

6 The alert (AL-0001)

Brute-force success • CRITICAL • pooja.sharma (user, Sales) • 2026-08-03 • risk 99.7

Why: 45 failed logins within 10 min followed by a successful login • Logins from locations too far apart to travel between • Login from 1 country never seen for this user • 400 MB sent to a domain this user never used before • 400 MB uploaded to file-sharing sites • 11 file accesses at hours this user is never active • 2 logins from a device not in the asset inventory • 401 MB uploaded vs usual 1 (388x usual) - also unusual vs peers

MITRE: T1110 Brute Force • T1078 Valid Accounts • T1048 Exfiltration Over Alternative Protocol • T1567.002 Exfiltration to Cloud Storage

Ground truth confirms INC-01. Detected the same day, ranked at the top of the alert queue.

SAY IT LIKE THIS

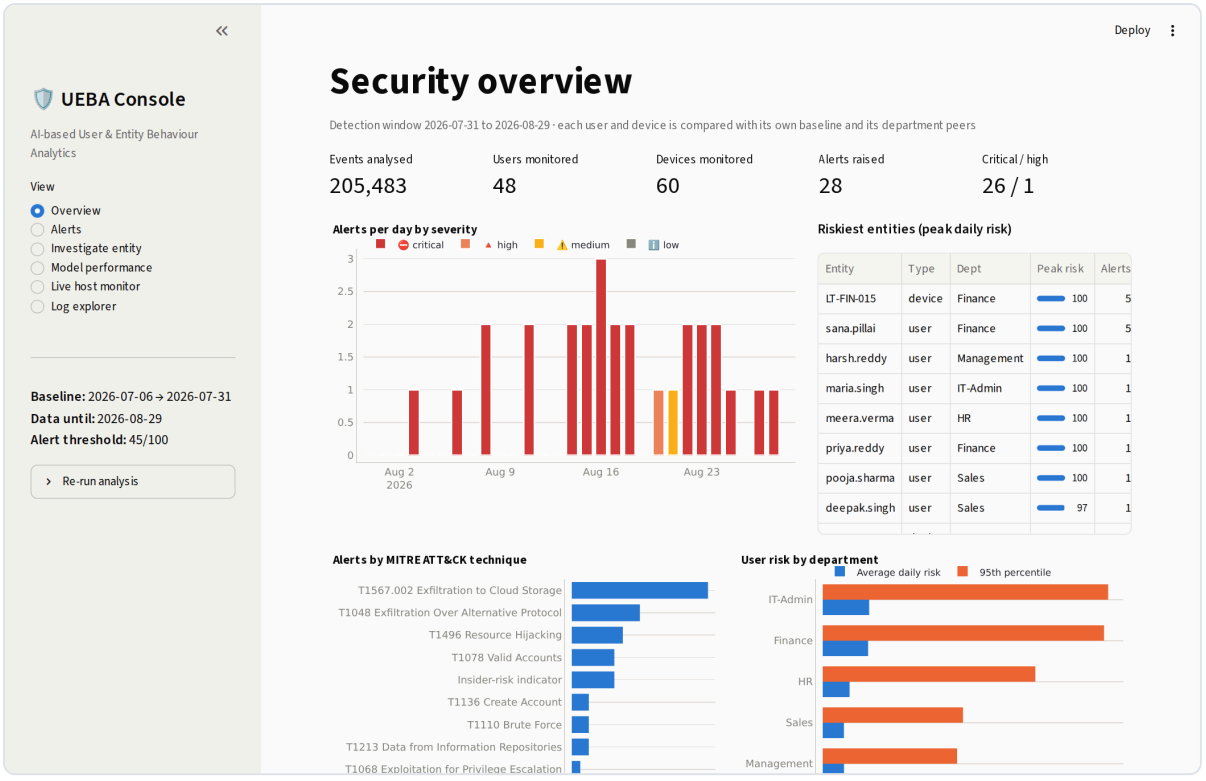
"Notice that the attacker used the real password, so a firewall would have seen nothing wrong. My system caught it because it knew Pooja never logs in at 3 a.m., never from Russia, never fails 45 times, and never uploads 400 MB."

SECTION 10

Dashboard tour

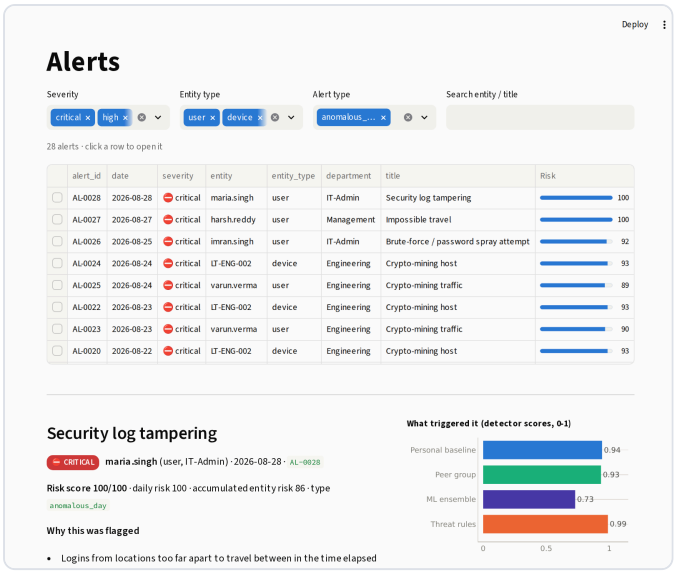
Built with Streamlit and Plotly. Six pages in the left sidebar, ordered the way a SOC analyst works: big picture → triage alerts → investigate one entity → check the model.

1 • Overview

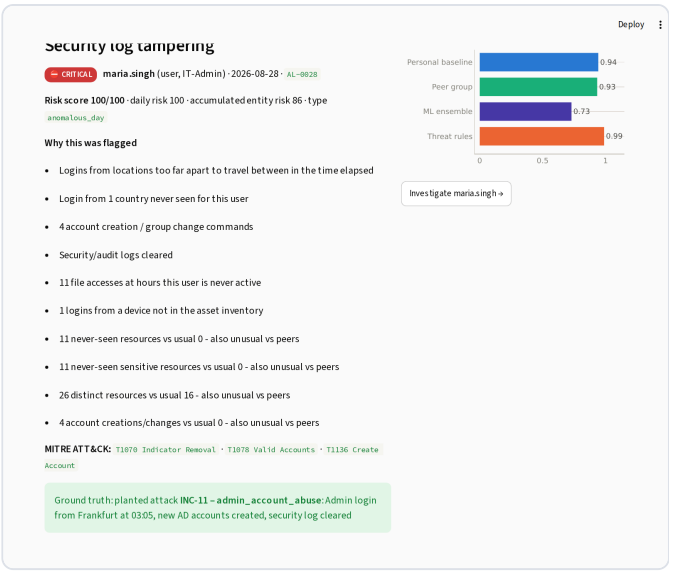


KPIs (events analysed, users and devices monitored, alerts, critical/high), alerts per day by severity, the riskiest entities, alerts by MITRE technique, and user risk by department.

2 • Alerts

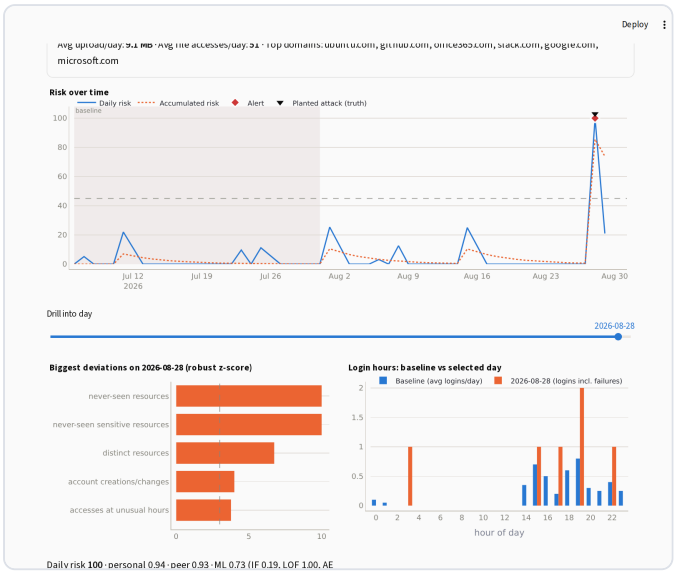


Alert queue: filter by severity, entity type and alert type, search by name, click a row to open it.

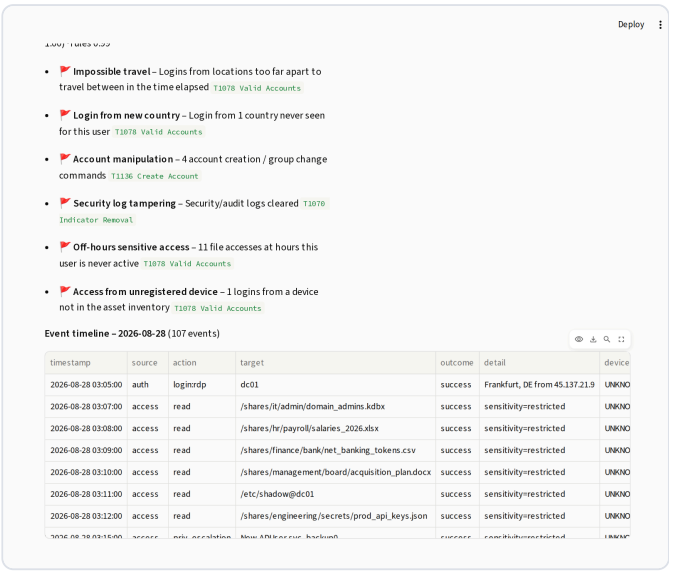


Alert detail: severity, risk, plain-English reasons, MITRE IDs, which layers fired, and the ground truth (green = real planted attack, yellow = false positive). Below the chart, **Mark benign** / **Confirm threat** buttons record the analyst's verdict: benign days are learned into the profile at the next run, confirmed threats never are.

3 • Investigate entity

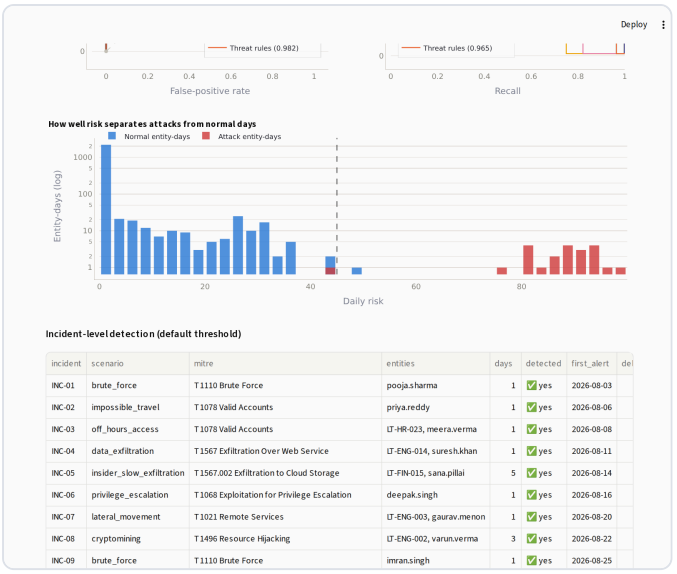


The learned baseline in words, plus risk over time: shaded baseline period, daily risk (blue), accumulated risk (orange, dotted), alert diamonds, black triangles for planted attacks.



Drill into any day: biggest deviations as z-score bars, login hours vs. baseline, the rules that fired, and the full event timeline.

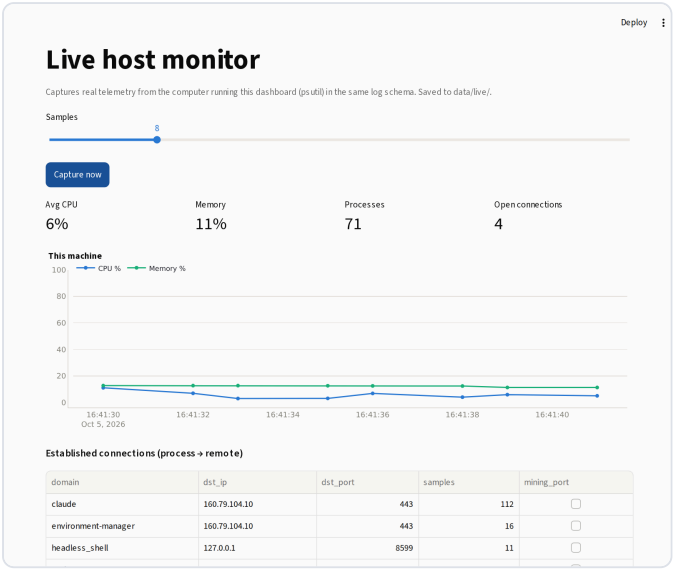
4 • Model performance



A what-if threshold slider that recomputes precision, recall, F1 and false-positive rate live, plus ROC and precision-recall curves per detector, risk distribution and an incident table.

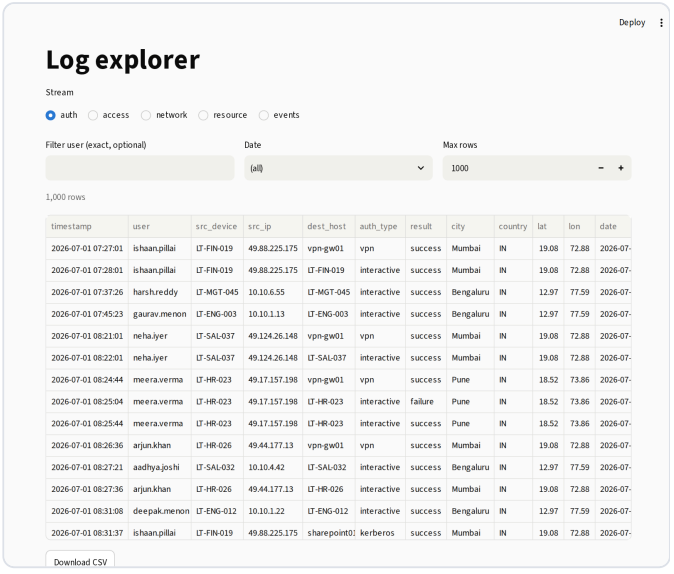
Use it to show the trade-off: drag the slider down to 30 and precision falls to about 0.5 (alert fatigue) while recall stays 1.0; at 45 all three metrics are 0.96.

5 • Live host monitor



"Capture now" samples the real CPU, memory, processes and open connections of the computer running the dashboard (here, the test server) in the same log schema, and flags known mining ports.

6 • Log explorer



Browse and filter any raw stream by user/device and date; download as CSV.

SECTION 11

Evaluation and results

Because every attack was planted, we know the right answer for every entity-day and can measure the system objectively. Measurement uses only the detection window (31 Jul - 29 Aug), never the training days.

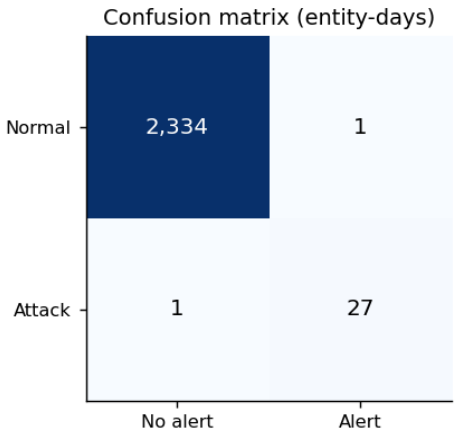
How to read the metrics

Metric	Formula	Plain meaning

Precision	TP / (TP + FP)	Of the alerts raised, how many were real attacks. Low precision = alert fatigue.
Recall	TP / (TP + FN)	Of the real attack-days, how many were alerted. Low recall = missed attacks.
F1	2PR / (P + R)	One number balancing both.
False-positive rate	FP / (FP + TN)	Share of normal entity-days wrongly alerted.
ROC-AUC	area under ROC curve	Chance that a random attack-day is ranked above a random normal day (1.0 = perfect ranking).
PR-AUC	area under precision-recall	Ranking quality that is honest when attacks are rare (28 of 2,363).

Main run (seed 42)

Metric	Value
Entity-days scored	2,363
Attack entity-days	28
Alerts raised	28 (~0.9 / day)
TP / FP / FN / TN	27 / 1 / 1 / 2,334
Precision / Recall / F1	**0.964 / 0.964 / 0.964**
False-positive rate	0.04 %
Incidents detected	**11 / 11**, all on day 1
Fused ROC-AUC / PR-AUC	1.000 / 0.999



The 1 false positive

diya.joshi (HR), 21 Aug, risk 49: opened 11 files she had never touched, from another department. This is the planted "new project" oddity. Genuinely unusual, harmless, and closed in seconds by an analyst.

The 1 false negative

LT-ENG-003, the *laptop* used as the pivot for lateral movement (INC-07), scored 43.6, just under 45. The *user account* doing the lateral movement was alerted the same day, so the incident was still caught.

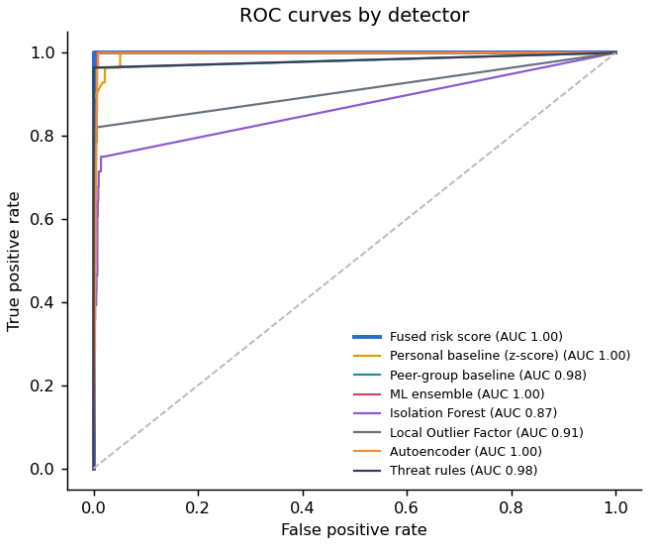


Figure 11.1 - ROC curves per detector.

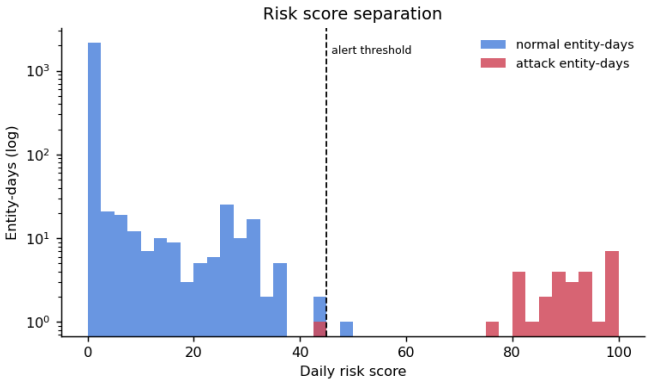


Figure 11.2 - Risk separates attack days (red) from normal days (blue); dashed line = threshold 45.

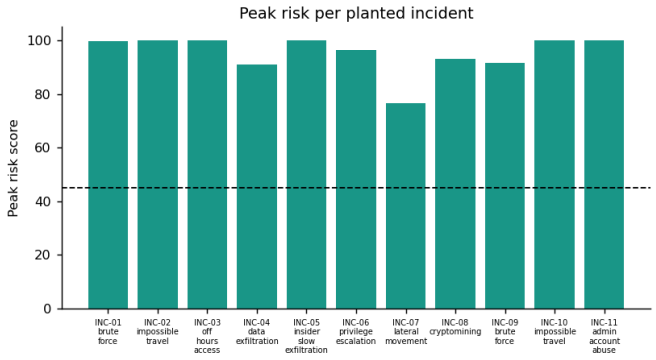


Figure 11.3 - Peak risk of each planted incident: all above the threshold.

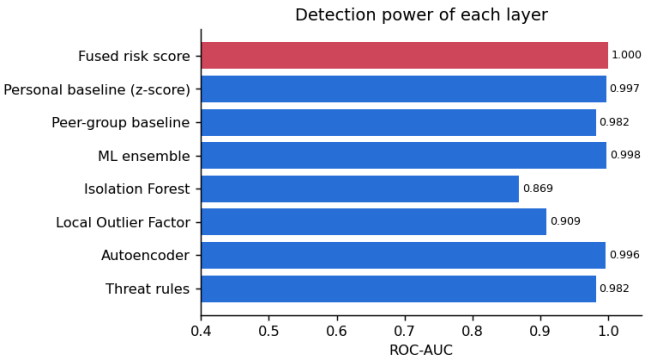


Figure 11.4 - ROC-AUC of each layer and each ML model.

Robustness: 5 different simulated companies

One good run could be luck. `experiments.py` regenerates the whole company with 5 random seeds: different people, habits, victims and timing.

Seed	Incidents	Precision	Recall	F1	FPR	Alerts/day
42	11/11	0.964	0.964	0.964	0.04%	0.93
7	11/11	0.931	0.964	0.947	0.09%	0.97
123	11/11	0.964	0.964	0.964	0.04%	0.93
2024	11/11	0.931	0.964	0.947	0.08%	0.97
99	11/11	0.900	0.964	0.931	0.13%	1.00
Mean	**55/55**	**0.938**	**0.964**	**0.951**	0.08%	0.96

Ablation: what each layer contributes

Switch layers off and re-measure (mean over the 5 seeds).

Variant	PR-AUC	Precision	Recall	F1
Full system	**0.996**	0.938	0.964	**0.951**
Without threat rules (behaviour only)	0.951	0.774	0.971	0.857
Without ML ensemble	0.984	0.979	0.964	0.971
ML ensemble only	0.856	0.763	0.821	0.789
Personal baseline only	0.457	0.295	0.971	0.451
Threat rules only	0.965	1.000	0.964	0.982

How to interpret this honestly (panels respect this)

- **Rules alone look near-perfect only because they were written knowing these 11 attack types.** Against a new attack with no rule, they score zero.
- **With all rules switched off, the behavioural layers still reach F1 0.86** with no knowledge of the attacks. That is the real value of UEBA, and of the "AI" in the title.
- Combining layers gives the best **ranking** (PR-AUC 0.996). In this synthetic data, removing ML slightly raises F1 because the rules already cover every planted attack; ML earns its place on attacks nobody wrote a rule for.
- Statistics alone flag everything unusual (recall 0.97 but precision 0.30). That is exactly why the corroboration threshold exists.

SECTION 12

Code map, tech stack, how to run

Which file does what

File	Responsibility
`run_pipeline.py`	Command-line entry point; runs everything and prints the evaluation
`ueba/config.py`	Every tunable number in one place (threshold, weights, window sizes, criticality)
`ueba/generator.py`	Simulated company, normal behaviour, 11 attacks, 4 harmless oddities, ground truth
`ueba/ingest.py`	Loads and validates logs (CSV/JSONL), unified event schema, SQLite read/write
`ueba/features.py`	44 user and 19 device features per day; novelty, unusual hour, impossible travel
`ueba/profiles.py`	Personal and peer baselines (median / MAD / P95), descriptive profiles
`ueba/detectors.py`	Statistical score, ML ensemble (IF, LOF, Autoencoder), 20 rules, reason text
`ueba/risk.py`	Risk fusion, criticality, accumulated risk with decay, alert generation
`ueba/pipeline.py`	Orchestrates all stages; adaptive baseline with quarantine
`ueba/evaluate.py`	Metrics, incident table, figures, markdown report
`ueba/collector.py`	Live CPU / memory / connection capture from the host (psutil)
`ueba/feedback.py`	Analyst verdicts (benign / threat) fed back into the baselines at the next run
`ueba/cert_loader.py`	Converts the CMU CERT insider-threat dataset into this schema
`dashboard/app.py`	Streamlit dashboard (6 pages)
`experiments.py`	5-seed robustness test and ablation study
`tests/`	11 automated tests: haversine, unusual-hour logic, rules, risk bounds, full pipeline, analyst feedback, CERT adapter

Tech stack and why

Tool	Used for	Why this choice
Python 3.10+	everything	required by the problem statement; standard for security data science
pandas, NumPy	logs and features	fast grouped aggregation over ~200k events
scikit-learn	IF, LOF, MLP autoencoder	well-tested implementations; no GPU needed
SQLite	storage	zero setup, single file, real SQL queries from the dashboard
Streamlit + Plotly	dashboard	interactive web UI in pure Python
Matplotlib	report figures	static PNG charts for documentation
psutil	live capture	cross-platform system telemetry
pytest	testing	automated checks before every demo

How to run (Mac / Linux / Windows)

```
cd "InHouse Project"
python3 -m venv .venv && source .venv/bin/activate    # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python run_pipeline.py          # ~30 s: logs → profiles → detection → alerts → evaluation
streamlit run dashboard/app.py  # opens http://localhost:8501

python -m pytest -q             # 11 tests
python experiments.py           # robustness + ablation (~3 min)
python run_pipeline.py --seed 7 --threshold 50    # different company / threshold
```

SECTION 13

Presenting: talk outline and demo script

Suggested 10-minute talk

Time	Topic	Key points (see section)
0:00-1:00	Problem	Signature tools miss stolen credentials and insiders; behaviour is the clue (§ 2)
1:00-2:00	Objective	Read the problem statement; the 4 requirements (§ 1)
2:00-3:30	Architecture	Show Figure 3.1; walk the 5 stages in one sentence each (§ 3)
3:30-5:00	Data	4 log streams, simulated company, 11 attacks + 4 harmless oddities and why (§ 4)
5:00-7:00	Method	Features → robust baselines → 3 detection layers → risk formula and threshold 45 (§ 5-8)
7:00-8:30	Results	11/11 detected, P/R/F1 0.96, 1 false alert; 5-seed robustness; honest ablation (§ 11)
8:30-10:00	Wrap-up	Limitations and future work (§ 15); then the live demo

Before the presentation (do this at home)

Run the pipeline once at home so `data/ueba.db` exists; the dashboard then opens instantly. Keep a second terminal ready to run `python -m pytest -q` live if asked. Don't rely on campus Wi-Fi: `pip install` needs the internet, so install beforehand.

5-minute live demo script

1. **Overview page (45 s)**
   "The system analysed about 205,000 events for 48 users and 60 devices. Here are the alerts per day, coloured by severity, and the riskiest entities on the right."

2. **Alerts → search "pooja" → open the brute-force alert (60 s)**
   "Critical, risk 99.7. Every alert explains itself: 45 failed logins then success, impossible travel, a new country, 400 MB to a file-sharing site. MITRE T1110. The bar chart shows all four layers agreed. The green box confirms it was a planted attack."

3. **Investigate entity → pooja.sharma (60 s)**
   "This is her learned profile: 9 to 6, Bengaluru, India only. Risk was flat through the baseline month, then a spike on 3 August. Drilling into that day, the login-hour chart shows activity at 3 a.m., when she has never worked. The timeline below shows the raw events."

4. **Investigate → sana.pillai (45 s)**
   "This is the slow insider: 5 days of growing Dropbox uploads plus job-site visits. The orange line is accumulated risk; it catches gradual behaviour that a one-day view could miss."

5. **Model performance, move the slider (60 s)**
   "At threshold 45: precision, recall and F1 all 0.96. If I lower it to 30, recall stays 1.0 but precision drops to about 0.5, which means alert fatigue. The ROC curves compare each detector."

6. **Live host monitor → Capture now (30 s)**
   "This captures real telemetry from this laptop right now, in the same log format, showing the capture stage works on live data, not only simulated logs."

If something breaks during the demo

Stay calm and switch to the screenshots in Section 10 of this guide. The figures in `reports/figures/` and the text report `reports/evaluation.md` are already generated.

SECTION 14

Viva questions and answers

Short answers you can say in 20-30 seconds. Understand them; don't memorise word for word.

Concept

Q. What is UEBA and how is it different from a SIEM or IDS?

A. A SIEM collects and searches logs; an IDS matches known attack signatures. UEBA learns the normal behaviour of each user and device and flags deviations, so it can catch attacks with no signature: stolen credentials, insiders, new techniques. In practice UEBA sits on top of SIEM data, as it would here.

Q. Why profile devices as well as users?

A. Some attacks show up on a machine rather than an account. Cryptomining malware pins a laptop's CPU; a laptop can be the pivot in lateral movement. 60 of the 108 profiled entities are devices.

Q. Why unsupervised learning instead of a classifier like Random Forest?

A. A classifier needs many labelled attacks, which real organisations don't have, and it only learns attacks it has seen. Unsupervised models learn only "normal" and flag anything different, including new attacks. Labels are used only to evaluate, never to train.

Q. Why three ML models instead of one?

A. They work on different principles: isolation (Isolation Forest), local density (LOF) and reconstruction error (Autoencoder). Each has blind spots; averaging calibrated scores is more stable. The ROC chart shows the Autoencoder and LOF are strongest here and Isolation Forest weaker, so the ensemble protects against relying on a weak one.

Q. Why median and MAD instead of mean and standard deviation?

A. Behaviour data has spikes. One 60 MB upload drags the mean and inflates the standard deviation, which hides real attacks. Median and MAD ignore a few outliers. I also take each person's 95th percentile into account so their normal occasional peaks aren't flagged.

Q. What is a z-score here, and why 3?

A. How many typical spreads today's value is from that person's normal. For normally distributed data, $|z| \geqslant 3$ happens about 0.3% of the time, a standard cut-off for "unusual". Scores are capped at 10 so one extreme feature cannot dominate.

Design

Q. How did you choose the weights and the threshold of 45?

A. The threshold has a design reason: statistics alone max out at 43.8, so an alert always needs ML or a rule to agree, which removes single-feature false positives. The weights give rules and ML more say than raw statistics. I then checked a threshold sweep: precision rises from 0.51 at 30 to 0.96 at 45 with recall unchanged. It is tunable in config.py and live on the dashboard.

Q. What is criticality and why multiply by it?

A. The same behaviour is more dangerous on a high-value account. A compromised domain admin can take over the company; a sales account cannot. So IT-Admin and servers × 1.3, Finance and Management × 1.2. It's standard risk thinking: risk = likelihood × impact.

Q. Couldn't an attacker slowly train your adaptive baseline to accept the attack?

A. That's baseline poisoning, and I handle it with quarantine: any day with risk $\geqslant 60$ or any rule hit is never added to the baseline. Only clean days are learned. Very slow, very low-level drift is still a known limitation of all UEBA systems.

Q. How do you catch slow attacks that look small each day?
A. Accumulated risk: daily risk above 15 is added to a running total that halves every 3 days. Consistent small anomalies build up; single odd days fade. Crossing 60 raises a sustained-risk alert.

Q. How is impossible travel calculated?
A. For consecutive successful logins, the great-circle distance between the two locations (haversine formula) divided by the time between them. More than 500 km at over 900 km/h, faster than an airliner, is flagged. Bengaluru to Amsterdam in 70 minutes is about 6,600 km/h.

Q. Why SQLite and Streamlit?
A. Zero setup and fully local: one database file, and a web dashboard written in pure Python. For a production system I would use Elasticsearch or a SIEM for storage and a streaming pipeline such as Kafka.

Data and results

Q. Your data is synthetic. Why should we trust the results?
A. Real logs never say which events were attacks, so synthetic data with planted attacks is the standard way to measure detection (CERT itself is synthetic). I made it realistic: individual habits, weekends, leave, typos, legitimate large uploads, plus 4 harmless oddities designed to cause false positives. I also tested 5 different random companies and built an adapter for the public CERT benchmark. I accept that real data would score lower; that is the main next step.

Q. Results of 0.96 look too good. Isn't it overfitted?
A. Three points. Nothing is trained on attacks; models only see normal days. The attacks are deliberately clear, as early-stage attacks often are, so the main challenge is false positives, and the harmless oddities test that. And I report honestly that rules look strong only because they were written for these attack types; without rules the system gets F1 0.86.

Q. Explain your false positive and false negative.
A. False positive: an HR user opened 11 files from another department, the planted "new project" case. Unusual but harmless; an analyst closes it in seconds. False negative: the laptop used for lateral movement scored 43.6, just under 45, but the user account was alerted the same day, so the incident was caught.

Q. Why precision and recall rather than accuracy?
A. Attacks are 28 out of 2,363 entity-days, about 1%. A system that never alerts would be 99% accurate and useless. Precision measures alert fatigue, recall measures missed attacks.

Q. What does the ablation show?
A. The full system gives the best ranking (PR-AUC 0.996). Behaviour layers alone, with no rules, still reach F1 0.86, which shows the system can detect attacks it was never told about. Statistics alone have high recall but poor precision, which justifies combining layers.

Q. What is the CERT dataset and did you use it?
A. Carnegie Mellon's public insider-threat benchmark: logon, USB, file, web and email logs for about 1,000 employees with labelled malicious insiders. I wrote and tested an adapter that converts it to my schema; running on the full 10-15 GB dataset is the next validation step.

Practical

Q. How would this work in a real company?
A. Replace the generator with real sources: Windows Security Event logs (4624/4625 logins), VPN and firewall logs, Sysmon or EDR process data, proxy logs. Map them to the same four CSV schemas; ingestion already accepts any CSV/JSONL with those columns. Then run daily, or stream events for faster detection.

Q. How fast is it?

A. The full pipeline, 205,000 events with all models, takes about 30 seconds on a laptop. Scoring is per entity-day, so it scales linearly with the number of users and days.

Q. What about privacy? You are monitoring employees.

A. Important point. UEBA should be deployed with a clear policy, monitoring only work accounts and devices, minimising data (we keep counts and metadata, not content), with access controls on the dashboard and compliance with laws such as India's DPDP Act 2023. Alerts go to trained analysts, not managers.

Q. What would you improve with more time?

A. Test on full CERT and real logs; real-time streaming; graph features for lateral movement; automatic response such as disabling an account or isolating a host via SOAR.

Q. What was the hardest part?

A. False positives. The first version caught everything but raised many false alerts, mostly on people's legitimate monthly uploads. Three fixes brought precision from about 0.3 to 0.96: the P95-aware spread, the adaptive baseline with quarantine, and the corroboration threshold. Fixing an uncapped accumulated-risk bug also removed lingering alerts after incidents.

Q. How do you know the code is correct?

A. 11 automated pytest tests: haversine distance, unusual-hour logic, each rule, risk-score bounds and the corroboration property, a full pipeline run that must catch the planted attacks, the analyst feedback loop, and the CERT adapter. Plus the 5-seed experiment.

SECTION 15

Limitations, future work, cheat sheet

Limitations (say them before the panel does)

- **Synthetic data** is cleaner than production logs; real-world precision would be lower.
- **CERT not yet run** on the full dataset (adapter built and tested on a sample).
- **Daily batch** scoring: detection comes at day level, not within minutes.
- **Rules cover known techniques** only; the behaviour layers cover the rest.
- **Very slow drift** below the quarantine level could still enter the baseline.

Future work

- Full CERT r4.2 / r6.2 evaluation and real Windows / Sysmon / Zeek logs.
- Streaming with Kafka, scoring each event within seconds.
- Graph analytics (user-host graph) for lateral movement.
- SOAR integration: auto-disable account, isolate host.
- Deep sequence models (LSTM / Transformer) over event order.

Cheat sheet: numbers to remember

4 log streams: auth, access, network, resource	**205k** events over 60 days	**48 + 60** users + devices = 108 entities	**44 / 19** features per user / per device per day
3 layers: statistics, ML, rules	**3 + 20** ML models + threat rules	**45** alert threshold (stats alone max 43.8)	**3 days** half-life of accumulated risk
11/11 incidents caught, all on day 1	**0.964** precision = recall = F1 (seed 42)	**0.951** mean F1 over 5 seeds	**0.86** F1 with all rules switched off

The whole project in five sentences

1. Collect four kinds of logs for every user and device.
2. Summarise each entity's day into 44 (or 19) behaviour numbers.
3. Learn what is normal for each entity and its department, with robust statistics that keep learning but quarantine suspicious days.
4. Score each day with three independent layers (statistics, an unsupervised ML ensemble, MITRE-mapped rules) and fuse them into a 0-100 risk adjusted for account criticality.
5. Raise an explained alert above 45, track slow attacks with decaying accumulated risk, and show everything in a dashboard built for security analysts.